Aster Support OS

Product Requirements Document — Stage 1

Version 1.0 | 29 October 2026 | Owner: Mudit Tandon

This document describes the current Stage 1 product and the requirements for testing it. It separates working features from future work. It does not replace the larger original specification in docs/specification.md or claim that the entire original specification is complete.

1. Product in one paragraph

Aster is a customer support workspace. Customers submit questions, support agents manage conversations, and administrators manage the team. A knowledge base helps agents find reliable answers. A local AI assistant can summarize a conversation and suggest an answer drawn from approved articles. Agents review suggestions before sending them. Workflows automate a limited set of internal actions, while audit records help explain what happened.

Stage 1 is a small, synthetic-data, non-commercial test release. It works locally, has passed integrated and container CI, and is prepared for a possible free-tier deployment. No public deployment has taken place.

2. Problem and opportunity

Support information often gets scattered across messages, documents and personal notes. Agents repeat the same answers, miss the history of a request, and struggle to see who owns the next step. Uncontrolled AI can add another problem by giving answers without evidence or attempting actions without permission.

Aster brings the request, conversation, reference articles, suggested response and action history into one place. Its first test should establish that this connected support journey is useful and understandable. It should not try to prove commercial scale, autonomous resolution or production reliability.

3. Users and responsibilities

User	Needs	Current responsibility
Administrator	Set up the workspace and supervise activity	Manage members/settings, review approvals and inspect audits
Support agent	Answer requests with context	Work through the inbox, maintain tickets, review drafts and reply
Customer	Ask for help and follow the response	Use the authenticated portal, view own requests, reply and give feedback
Project owner/tester	Evaluate the product safely	Use synthetic fixtures, review tests and monitor release limits

An organization is one team's workspace. Records belong to that organization. The current membership model links a user to one organization; multi-organization switching and custom permission builders are future work.

4. Goals and boundaries

The release must demonstrate one complete journey: a customer asks a question, an agent receives it, the agent finds a cited suggestion, reviews and sends a response, and the customer can see the response and

give feedback. The saved record and dashboard must reflect the actual actions.

Additional goals are to keep teams' data separate, keep manual support working during an AI outage, recover after process restarts, preserve data through migrations, and run the test without paid inference or paid infrastructure.

Stage 1 does not include live email or WhatsApp delivery, real billing/refunds, attachment uploads, semantic vector search, unlimited autonomous tools, production SLA guarantees, or certification. A settings page or approval record is not evidence that an external integration executes.

5. Primary user journeys

5.1 Start a workspace

The administrator creates an organization with a name, full name, email and password. A successful sign-in opens the workspace. Invalid or duplicate registration must produce an understandable error. Passwords are hashed; the browser uses an HttpOnly session cookie. Invitations determine whether a new member joins as an agent or customer.

The public-beta configuration allows synthetic test email addresses and enforces limits. The local development configuration may expose clearly labeled sandbox features that are disabled in public-beta mode. These are separate operating profiles.

5.2 Handle a customer request

A customer submits a subject and description through the portal. The agent sees the ticket in the inbox and can assign it, change priority, add labels, write a private note and update its status. The customer sees public replies and their own history, but cannot see another customer's requests or staff-only notes. A solved request can collect satisfaction feedback.

5.3 Use an evidence-based suggestion

Staff add current knowledge articles and choose team/public visibility. Aster divides article text into searchable pieces, retrieves permitted text for the ticket, and returns an extractive suggestion with references. If useful evidence is missing, it should abstain. The agent reviews the text. Before a saved proposal is sent, Aster rechecks the source's permission, freshness and version.

The default assistant uses deterministic extraction and rules, not a paid generative model. It does not provide semantic search or guarantee the right answer. If the Python service is unavailable, manual conversation handling remains available.

5.4 Automate a bounded internal action

An administrator builds a workflow, checks it in simulation, publishes a version, then observes its run history. Supported ticket events can trigger bounded internal actions. A failing action rolls back its effects while leaving a failure record. A human-review node creates an awaiting-human checkpoint. Automatic continuation after approval and durable timers are not implemented.

6. Functional requirements and acceptance

The labels below describe the current implementation, not the full original roadmap. "Verified" means covered by the local/CI evidence for this release, rather than exhaustively proven for every possible input.

ID	Requirement	Acceptance example	Current state
FR-01	Organization identity and role checks	A customer cannot perform an administrator action	Verified
FR-02	Tenant separation	Organization A cannot read or update B's ticket or knowledge	Verified
FR-03	Inbox and conversation	Create, assign, label, prioritize, search, reply and change ticket state	Verified
FR-04	Private staff notes	A customer response excludes internal notes	Verified
FR-05	Knowledge lifecycle	Article visibility, freshness and archive state affect retrieval	Verified
FR-06	Chunk retrieval and citations	Returned references identify permitted source text	Verified; lexical, not semantic
FR-07	Safe proposal dispatch	Changed/stale sources prevent an old proposal being sent unchanged	Verified
FR-08	AI fallback	AI outage does not prevent a manual reply	Verified
FR-09	Local tools and approvals	Unsupported shell tools are rejected; sensitive external actions do not execute	Verified boundary; external adapters absent
FR-10	Workflow editing and history	Draft, validate, simulate, publish, run and inspect a saved version	Verified for supported nodes
FR-11	Event-triggered workflows	Supported ticket events run bounded published workflows	Verified; no durable timers/resume
FR-12	Dashboard and analytics	Counters derive from stored activity; solved is not called independently verified resolution	Verified for implemented metrics
FR-13	Customer experience	Authenticated request/reply/history/CSAT and searchable public help content	Verified
FR-14	Audit and notifications	Actions leave records; notification projection is idempotent	Verified locally; not Kafka consumer proof
FR-15	Release safety flags	Public beta rejects real-email use, sandbox access and paid inference	Verified
FR-16	Responsive UI	Principal desktop/mobile, theme, keyboard and accessibility flows work	Verified tested screens; no full WCAG certification
FR-17	Live external providers	Send real email/WhatsApp or execute payment changes	Not implemented
FR-18	Production operational controls	Restore drills, retention, tracing/alerts and sustained-load validation	Incomplete/unverified

7. Screens and information architecture

Screen	Purpose
Landing and authentication	Explain the product; sign in, create an organization or join by invitation
Overview/dashboard	Show current support activity and entry points
Inbox and conversation	Work on tickets and review contextual assistance
Customers	Inspect customer context within the organization

Screen	Purpose
Knowledge	Maintain reference articles and review ingestion/retrieval state
Workflows	Edit, simulate, publish and inspect supported workflows
Approvals	Inspect actions requiring administrator review
Analytics and AI activity	Understand real recorded activity and assistant mode/outcomes
Integrations and billing	Show honest configuration/trial state; no live delivery or charging claim
Team/settings, audit and sandbox	Manage the workspace and inspect records or local-only simulations
Customer portal, help and widget	Provide authenticated self-service and public articles

UI requirements include understandable loading, empty, success and error states; readable contrast; dark/light themes; mobile navigation; reduced-motion support; and keyboard alternatives. No metric should be fabricated to make a demo look active. Demo records must be labeled synthetic.

8. Technical design in plain language

The Next.js frontend is the part users see. Its same-origin API proxy forwards requests to the Rust/Axum service, which owns access checks and business rules. PostgreSQL stores the durable records. The internal Python/FastAPI service handles local analysis and text extraction. Python does not independently authorize a public user's data access.

The ordinary local Compose stack runs the frontend, Rust, Python and PostgreSQL as separate services. The constrained beta layout combines Rust and Python under one supervisor to rehearse a single small backend instance. Kafka is optional and is not needed for the tested Stage 1 journey.

The database uses additive migrations: startup applies missing schema changes while retaining existing rows. Nine migrations were verified. An outbox records events together with their associated data changes; it does not prove delivery to an external provider.

9. Security, privacy and reliability requirements

Access must be checked against authenticated organization and live role information. Requests must be bounded by body/rate limits. Duplicate action keys must not silently duplicate effects. User-facing errors must avoid database secrets. Model or service outages must leave manual support available.

The tested containers run as non-root users. The local rehearsal additionally verifies read-only roots, no effective capabilities, no-new-privileges and removal of SUID/SGID programs. A future host must be checked independently; Compose settings do not prove equivalent cloud controls.

Raw image vulnerability matches remain, including critical package matches in the PostgreSQL image. The current JSON/TEXT application paths and exact helper binary were assessed for this limited build. This is not a claim that those libraries are patched. Unknown findings or source/config/dependency drift must reopen review. Real customer data and commercial/production use are outside this approval scope.

10. Cost and performance requirements

The budget target is ₹0/month for an eligible, small non-commercial test within provider quotas. The prepared topology is a Vercel Hobby frontend, one Render Free combined backend and Neon Free

PostgreSQL. This is a project plan, not a guarantee of current provider eligibility, quotas or pricing. Recheck official terms and the actual account before any future deployment. No resource or paid account was created.

The constrained backend was tested at 0.1 CPU and 512 MiB. Idle polling is reduced, the connection pool is bounded, requests have timeouts and the inbox is paginated. Small synthetic measurements are not a capacity estimate. No production availability, p95 latency or resolution-cost SLO is approved; baseline measurements and hosted tests must come first.

11. Release evidence and definition of done

Verified application commit: d83daf1c50dc204493889560156175d081cc5798 on release/stage1-testing. GitHub workflow #9 completed successfully: https://github.com/MuditTandon20/customer_support_platform/actions/runs/40900733408. Both integrated and container jobs passed.

Check	Recorded result
Rust API unit tests	5 passed
Isolated Rust advisory reproduction	1 passed
Python/configuration/assessment unit tests	22 passed
Full-stack API suites	18 passed: 11 existing plus 7 Stage 1
Constrained beta API suites	7 passed
Browser tests	8 passed
Schema, health and recovery	Nine migrations; service/database outages; stop/start; recreation and supervisor recovery passed
Frontend	Lint, TypeScript, production build and npm audit passed
Security assessment	Current scoped gate passed; zero unreviewed image findings and zero detected affected gosu symbols

A requirement is complete only when the relevant backend/UI behavior, permissions, failure handling and evidence exist. Passing this test release does not make a future provider adapter, production launch or full original PRD complete.

12. Success measures for the first test

Observe whether a tester can complete the support journey without help, whether the reply and source are understandable, and whether organization/customer boundaries hold. Record completion/failure counts, actual response timings, citation usefulness, manual takeover and feedback from testers. Treat these as measurements to collect; no adoption, revenue, savings or resolution improvement has yet been established.

13. Remaining work and launch decision

Before an authorized hosted test, verify account eligibility and quotas, actual container privileges, TLS, secure cookies, exact origin, secrets, database access restrictions and beta flags. Exercise the hosted end-to-end journey and record cold-start behavior. Stop if free hosting cannot meet required controls; keep the local test instead.

Later implementation may add production channels, signed webhooks, safe attachments, semantic retrieval, durable workflow continuation, real billing, custom permissions and operational controls. Their order is a future product decision, not a promise that the features exist or a dated delivery commitment.

Current decision: eligible for synthetic, non-commercial Stage 1 testing of the reviewed build. Not deployed. No authorization exists here to deploy, spend money, force-push or merge into main.

14. Source of truth

Read alongside docs/STAGE1_SECURITY_CLOSURE.md, docs/STAGE1_SECURITY_REVIEW.json, docs/STAGE1_RUNTIME_SECURITY.json, docs/STAGE1_RELEASE_REPORT.md, docs/requirements-traceability.md and the original docs/specification.md. Older reports contain historical counts and superseded holds; the green workflow on the exact reviewed application commit is the latest executed CI evidence.